

MoE-APEX: An Efficient MoE Inference System with Adaptive Precision Expert Offloading

Peng Tang*
Shanghai Jiao Tong University
Shanghai, China
ttppp@sjtu.edu.cn

Jiacheng Liu*
The Chinese University of Hong Kong
Hong Kong, China
liujiacheng@ieee.org

Xiaofeng Hou†
Shanghai Jiao Tong University
Shanghai, China
hou-xf@cs.sjtu.edu.cn

Yifei Pu
Shanghai Jiao Tong University
Shanghai, China
pkq2006@sjtu.edu.cn

Jing Wang
Shanghai Jiao Tong University
Shanghai, China
jing618@sjtu.edu.cn

Pheng-Ann Heng
The Chinese University of Hong Kong
Hong Kong, China
pheng@cse.cuhk.edu.hk

Chao Li†
Shanghai Jiao Tong University
Shanghai, China
lichao@cs.sjtu.edu.cn

Minyi Guo
Shanghai Jiao Tong University
Shanghai, China
guo-my@cs.sjtu.edu.cn

Abstract

Mixture-of-experts (MoE) architectures enable scalable Large Language Models (LLMs) with reduced computational overhead, yet their deployment on memory-constrained edge devices is hindered by substantial memory demands. Traditional expert-offloading techniques mitigate memory constraints but often significantly increase inference latency. We introduce MoE-APEX, an Adaptive Precision EXpert offloading system that optimizes MoE inference for edge architectures by dynamically managing expert precision. Our core innovation is to replace less critical cache-miss experts with low-precision variants, reducing loading latency while maintaining accuracy. MoE-APEX introduces three innovative techniques that map the natural hierarchy of MoE computation: (1) a token-level dynamic expert loading mechanism, (2) a layer-level adaptive expert prefetching technique, and (3) a sequence-level cost-aware expert caching policy. These innovations enable MoE-APEX to leverage the benefits of mixed-precision expert inference fully. Implemented atop Llama.cpp, MoE-APEX achieves decoding speedups ranging from 1.34× to 9.75× compared to state-of-the-art MoE offloading systems across diverse edge devices, offering a robust solution for efficient MoE deployment in resource-constrained environments.

*Both authors contributed equally to this research.

†Co-corresponding authors.



This work is licensed under a Creative Commons Attribution 4.0 International License.

ASPLOS '26, Pittsburgh, PA, USA.

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2359-9/2026/03

<https://doi.org/10.1145/3779212.3790187>

CCS Concepts: • Computer systems organization → Real-time system architecture.

Keywords: Edge Computing; Inference Acceleration; Parameter Offloading

ACM Reference Format:

Peng Tang, Jiacheng Liu, Xiaofeng Hou, Yifei Pu, Jing Wang, Pheng-Ann Heng, Chao Li, and Minyi Guo. 2026. MoE-APEX: An Efficient MoE Inference System with Adaptive Precision Expert Offloading. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '26)*, March 21–26, 2026, Pittsburgh, PA, USA. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/3779212.3790187>

1 Introduction

The rapid proliferation of Large Language Models (LLMs) has revolutionized diverse application domains [65]. Beyond deploying LLM in cloud-based data centers, edge deployment of LLMs is increasingly crucial to address inherent limitations of centralized approaches, including high latency, privacy vulnerabilities, and network dependency [17]. Consequently, enabling edge deployment of LLMs has emerged as a critical research focus, with both academia [6, 59, 63] and industry [4, 24, 44] actively working to accelerate the adoption of LLMs on resource-constrained edge devices.

Specifically, MoE replaces the traditional MLP module with a MoE module in the transformer architecture. However, MoE-based LLMs demand substantial GPU memory for parameter storage. For instance, the Mixtral-8x7B model [27], despite activating only 14 billion parameters per token, requires 87GB of memory to store its complete set of 45 billion parameters. This poses significant deployment challenges on memory-constrained edge devices, such as the NVIDIA

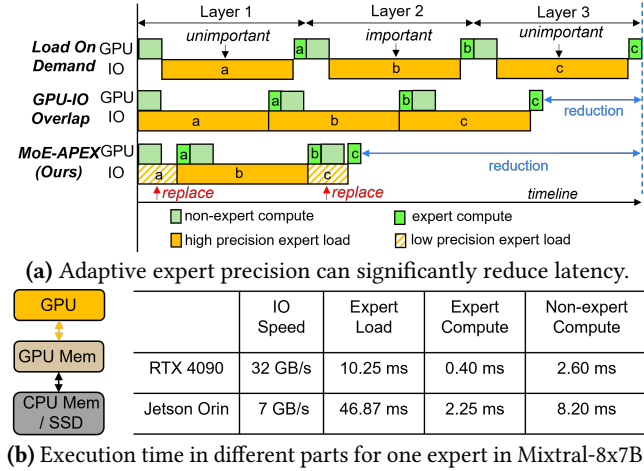


Figure 1. MoE inference timeline and execution costs.

Jetson AGX Orin with its 32GB memory capacity. Expert-offloading techniques address this limitation by keeping only part of the parameters in memory, leveraging the sparse activation patterns inherent in MoE.

The core principle behind expert-offloading systems involves maintaining all non-expert weights and a subset of critical experts in GPU memory (the "expert cache"), while relegating remaining experts to CPU memory or SSD (the "next-level memory"). However, as illustrated in Figure 1-(a), traditional approaches primarily rely on load-on-demand and GPU-IO overlap, which fail to fully address latency challenges due to significant loading delays that cannot be completely hidden by computation overlap. Figure 1-(b) further quantifies these issues, showing that loading a single expert (336MB in float16) on a memory-constrained device, such as the Jetson Orin, is approximately 20× slower than GPU computation and 5× slower than non-expert processing.

To fundamentally address this performance bottleneck, we propose a novel adaptive precision approach for expert loading. Our key insight is that not all experts contribute equally to model outputs, making it possible to selectively replace less critical experts with low-precision variants during cache misses. This approach promises significant reductions in loading time while maintaining model accuracy. However, implementing adaptive precision expert loading introduces several fundamental challenges that require systematic redesign across the entire MoE inference stack:

Dynamic Expert Importance Assessment. Determining which experts can be safely loaded in lower precision represents a critical challenge in adaptive precision systems. Existing approaches either rely on offline static profiling to determine expert bit-widths (EdgeMoE [60], MC-MoE [22]), lacking flexibility across diverse inputs, or aggressively skip experts (AdapMoE [66]), causing accuracy degradation with small top-k values. An online dynamic

mechanism is needed to assess expert importance at runtime and make fine-grained precision decisions without adding significant computational overhead.

Optimized Prefetching for Mixed-Precision Experts.

Conventional prefetching techniques face substantial challenges in the context of mixed-precision experts. Existing methods like MoE-Infinity [58], MoE-Offloading [13], and Pre-gated MoE [25] attempt to predict which experts will be needed in subsequent layers, but achieve limited benefits because they fail to account for the significant imbalance between expert-loading cost and GPU computation time. In an adaptive precision context, the challenge becomes even more complex, requiring greater foresight to fully leverage the benefits of adaptive precision expertise.

Precision-Aware Cache Management. Traditional cache replacement policies are ill-suited to handle the unique characteristics of mixed-precision expert caching. For instance, the least frequently used (LFU) policy, employed in previous works [58, 60], tracks the usage frequency of each expert but overlooks the varying loading costs associated with high- and low-precision experts. This results in suboptimal performance when loading experts of different precisions.

In response to these challenges, we present MoE-APEX, a system designed to accelerate expert loading across three levels of MoE computation. As shown in Figure 1-(a), MoE-APEX significantly accelerates MoE-based LLM inference on memory-limited devices by dynamically replacing unimportant experts with low-precision versions. Our architecture maps directly to the natural hierarchy of MoE computation: at the token level, a Dynamic Expert Loader assesses importance through gating outputs; at the layer level, an Adaptive Expert Predictor leverages similarity between consecutive layers for efficient prefetching; and at the sequence level, a Cost-aware Cache Manager implements optimized caching policies. These modules work in concert to minimize expert loading latency while maintaining model accuracy, enabling efficient deployment of large-scale MoE models on resource-constrained edge devices. The key contributions are summarized as follows:

- We propose a token-level dynamic expert loading mechanism that reduces latency through low-precision replacement of less critical cache-miss experts, maintaining accuracy and flexibility.
- We introduce a layer-level adaptive expert prefetching technique with high prediction accuracy, leveraging mixed-precision prefetching to optimize computation-communication overlap.
- We develop a sequence-level cost-aware expert caching policy that combines model-specific locality characteristics with mixed-precision features to efficiently manage the expert cache and minimize miss penalties.
- We implement MoE-APEX on top of Llama.cpp with 8,500 additional lines of C++/C code, and evaluate it on

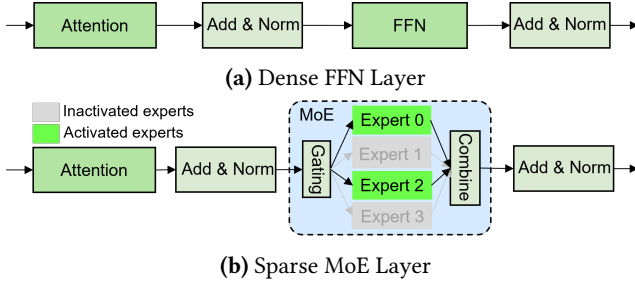


Figure 2. Comparison between different LLM architectures.

four popular MoE-based LLMs across three memory-limited platforms, demonstrating up to 9.75× speedup in decoding over state-of-the-art systems.

2 Background and Motivation

2.1 Background

Sparse MoE Layer. Due to the effectiveness of the MoE architecture [26], numerous MoE-based models [12, 21, 51] have emerged. In this work, we focus on the most widely used sparse MoE layer [45], which employs multiple FFNs (Feed-Forward Networks) as experts. As shown in Figure 2, unlike dense layers, the MoE layer uses a gating function to select the K most relevant experts (2 in the figure) for each input token, aggregating their outputs. For an input x , the output y of the MoE module can be formulated as:

$$y = \sum_{i=1}^K G(x)_{e_i} E_{e_i}(x) \quad (1)$$

where e_i is the i -th selected expert in the current layer, $G(x)_{e_i}$ represents the gating weight of expert e_i , and $E_{e_i}(x)$ is the output of expert e_i . The gating function $G(x)$ is typically implemented using a linear layer followed by a Top- k operation [10, 15, 27, 50].

Expert Offloading. Parameter-offloading techniques typically transfer part of the model’s parameters to CPU memory or SSDs when GPU memory is insufficient [3]. However, most offloading systems, such as Zero-Infinity [43] and Accelerate [20], are designed for dense LLMs and load model parameters layer-by-layer on demand. This approach overlooks the sparse activation nature of MoE models, resulting in substantial latency. For instance, loading a layer (approximately 2.7 GB) of the Mixtral-8x7B model from CPU memory via a PCIe 4.0 link (32GB/s) takes approximately 80ms, while computing it on an RTX 4090 GPU requires only about 3ms.

To address this issue, some studies have developed expert-offloading, a specialized form of parameter-offloading tailored to the sparse activation characteristic of MoE [13, 28, 58]. As shown in Figure 3-(a), this technique typically considers two levels of hardware memory: GPU memory stores all non-expert weights, a subset of “hot experts” (expert cache), and internal activations, while other experts are offloaded to

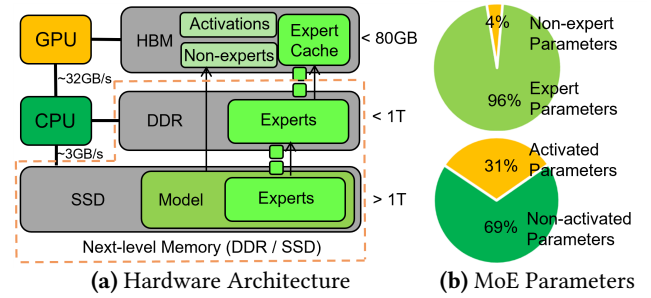


Figure 3. Expert-offloading on hardware architecture and model parameter distribution for Mixtral-8x7B.

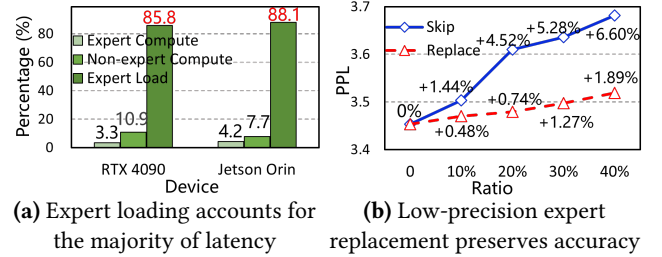


Figure 4. Analysis of expert loading acceleration chances.

CPU memory or SSD and loaded on demand. This approach is effective because of the sparse activation pattern in MoE models, where each token requires all non-expert weights but only a fraction of the experts. As illustrated in Figure 3-(b), non-expert weights account for only 4% of the Mixtral-8x7B model, and only 31% of the parameters are activated per token. Despite the effectiveness, existing expert-offloading techniques still incur high latency due to on-demand loading. While some of the works focus on optimizing prefetching techniques and cache replacement policies to accelerate inference speed, they remain constrained by the significant cost of expert loading during cache misses.

2.2 Motivations

We identify two key observations that motivate our work: **Expert loading dominates inference cost.** To quantify the bottlenecks in MoE model inference, we measured the time costs of different operations when running a Mixtral-8x7B layer on two memory-limited edge devices: an RTX 4090 (representing an edge server) and a Jetson Orin (representing an end device). As shown in Figure 4-(a), expert loading dominates the total inference time, consuming approximately 85.8% on the RTX 4090 and 88.1% on the Jetson Orin, while computation accounts for only a small fraction. While prefetching is commonly used to accelerate offloading by overlapping computation with data loading, its benefits are severely limited in MoE models due to this disproportionate time distribution. Some researchers have attempted to

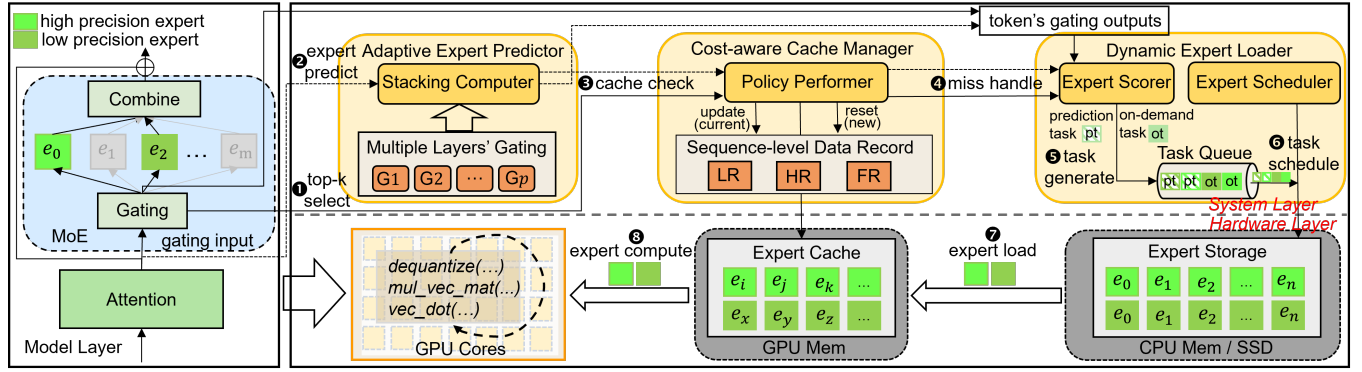


Figure 5. System overview of MoE-APEX.

address this by employing dynamic gating to limit the number of experts loaded [33, 66]. However, this approach comes with significant accuracy trade-offs. As shown in Figure 4-(b), the "Expert Skip" method results in notable degradation of model performance, with a 10% expert skip rate causing more than a 1% increase in perplexity (PPL).

Mixed precision expert preserves model accuracy. Quantization is an effective method for reducing model parameter size, but directly quantizing the entire model can result in substantial accuracy loss. In MoE models, different experts have varying levels of importance [30, 60, 66], so quantizing only the less important experts minimally impacts accuracy. As shown in Figure 4-(b), compared to skipping some experts, replacing them with low-precision versions better maintains model accuracy, and the gap between skipping and replacing grows as the ratio increases. In particular, when fewer than 20% of the experts are quantized, model performance declines by no more than 1%. Thus, applying quantization to low-importance experts in expert-offloading techniques can significantly reduce expert-loading cost. Specifically, if a required expert is not available in GPU memory and its importance is low, we can fetch a lower-precision version to replace it, thereby greatly reducing loading time. For instance, replacing a float16 expert with an int2 version can achieve up to a 8× speedup in the loading process.

These observations motivate the need for a system that can dynamically manage expert precision during inference while maintaining model accuracy.

3 MoE-APEX System

3.1 Overview of MoE-APEX

MoE-APEX is an **Adaptive Precision EXpert** offloading system designed for the inference of MoE-based LLMs on memory-limited devices. It incorporates three-level innovations: (i) a token-level dynamic expert loading mechanism that selects the appropriate precision expert from CPU memory or SSD; (ii) a layer-level adaptive expert prefetching technique

that provides highly accurate prefetching decisions for subsequent layers; and (iii) a sequence-level cost-aware expert caching policy that explores the locality characteristics of MoE models along with the unique features of the mixed precision experts. As shown in Figure 5, MoE-APEX consists of three main modules built upon these mechanisms: Dynamic Expert Loader, Adaptive Expert Predictor, and Cost-aware Cache Manager. The three-level design of MoE-APEX directly maps to the natural hierarchy of MoE computation, ensuring comprehensive optimization.

When executing a MoE layer on the GPU, the system first ❶ selects the top- k required experts (referred to as on-demand experts) for MoE computation based on the gating outputs. Simultaneously, the Adaptive Expert Predictor ❷ predicts the experts needed for subsequent layers (referred to as prediction experts) using its Stacking Computer, based on the current gating input. The Cost-aware Cache Manager then ❸ checks if the required experts are present in the expert cache and updates (for the current processing sequence) or resets (for a new coming sequence) the data record with its Policy Performer. If all on-demand experts are present in the cache, ❹ the expert computation is performed on GPU cores.

If any on-demand or prediction experts are missing from the cache, the Dynamic Expert Loader uses the Expert Scorer to ❺ handle the cache miss based on the gating outputs of the current processing token. The Expert Scorer dynamically ❻ generates the corresponding loading tasks with varying precision requirements, adding them to the Task Queue. The Expert Scheduler module in the Dynamic Expert Loader ❼ then fetches tasks from the Task Queue and ❽ loads the corresponding experts from the Expert Storage into the Expert Cache. If necessary, the Cost-aware Cache Manager will replace older experts in the cache based on the proposed caching policy. The system waits for all on-demand expert loading tasks to complete before ❾ computing the outputs of the experts for the MoE module and advancing to the next layer. This process efficiently handles expert cache misses and accelerates inference by reducing expert-loading costs through the use of adaptive precision experts.

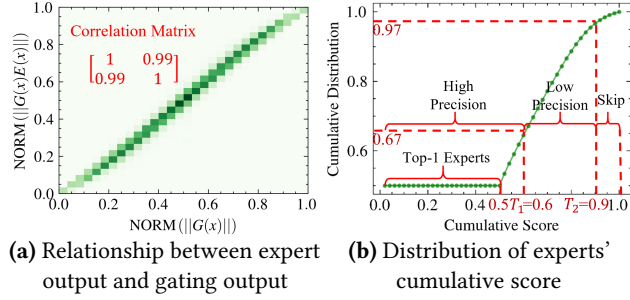


Figure 6. Gating output statistics of Mixtral-8x7B.

3.2 Token-level Dynamic Expert Loading

Loading low-precision experts during cache misses effectively mitigates expert loading latency, as demonstrated in Section 2.2. However, to preserve model accuracy, this replacement should target only less important experts. While model profiling on specific datasets can identify expert importance, this static approach is impractical for diverse deployment environments. Instead, we need a dynamic method to assess expert importance based on runtime inputs during the LLM’s generation process.

Expert importance estimation. Based on the computing pattern of the MoE module in Equation (1), expert e_i contributes $G(x)_{e_i}E_{e_i}(x)$ to the output y . We can represent the influence of expert e_i on the output using the magnitude $\|G(x)_{e_i}E_{e_i}(x)\|$ (where $\|\cdot\|$ denotes magnitude), as a smaller magnitude implies that the values in the tensor are closer to zero. Since $E_{e_i}(x)$ cannot be computed without the weight of expert e_i , we approximate $\|G(x)_{e_i}E_{e_i}(x)\|$ using $\|G(x)_{e_i}\|$. This approximation is based on our observation that $\|G(x)_{e_i}\|$ and $\|G(x)_{e_i}E_{e_i}(x)\|$ are positively correlated. To confirm this positive relationship, we collected both the expert output $\|G(x)E(x)\|$ and the gating output $\|G(x)\|$ from the Mixtral-8x7B model. After normalizing the data, we compute the Pearson correlation coefficient matrix and plot a heatmap to visualize their relationship. As shown in Figure 6-(a), the two variables exhibit a strong positive correlation, with a coefficient of 0.99.

Takeaways: We can leverage $\|G(x)\|$ as a computationally efficient proxy for expert importance, given its strong positive correlation with $\|G(x)E(x)\|$.

Expert loader design. Based on the observations above, we first rank the selected K experts in descending order of $\|G(x)_{e_i}\|$ (where a larger i corresponds to a smaller $\|G(x)_{e_i}\|$, and $\|G(x)\|$ values are normalized). Next, we calculate the cumulative score s_{e_i} for each expert e_i as follows:

$$s_{e_i}(x) = \begin{cases} \sum_{j=0}^{i-1} \|G(x)_{e_j}\|, & i > 0 \\ 0, & i = 0 \end{cases} \quad (2)$$

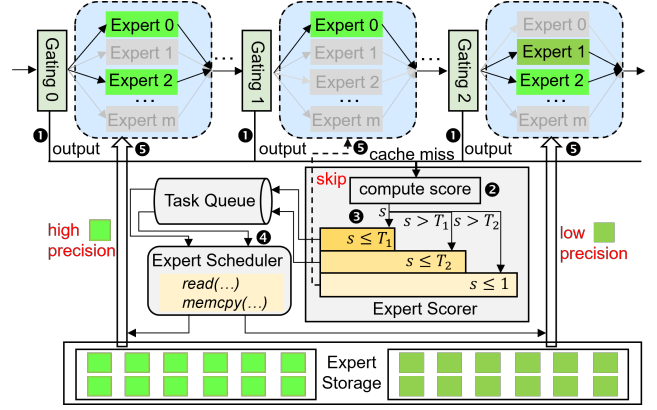


Figure 7. Token-level Dynamic Expert Loader.

where x is the gating input of current processing token. This score will determine whether the expert is replaced with a low-precision version. Specifically, we set a threshold T_1 (where $0 \leq T_1 \leq 1$): if $s_{e_i} \leq T_1$, we consider the expert important and load the high-precision version; otherwise, we opt for the low-precision version to reduce loading overhead due to its minimal influence on the output. Notably, we always treat the first expert (e_0) as important, keeping it in high precision to maintain model accuracy.

Based on the cumulative score, we implement the Dynamic Expert Loader as illustrated in Figure 7. To increase flexibility, we introduce a second threshold T_2 , allowing the system to bypass less important experts. As shown in Figure 7, when ① a cache miss occurs, the Expert Scorer module ② computes the scores of the missed experts and generates appropriate tasks based on these scores, ③ adding them to the Task Queue. The Expert Scheduler then ④ fetches tasks from the queue and ⑤ loads the corresponding precision experts from expert storage via system calls, such as `read(...)`. For instance, in the figure, Gating 0 retrieves a high-precision expert due to its high importance, Gating 1 skips an expert deemed of very low importance, and Gating 2 fetches a low-precision expert for moderate importance. To select the threshold values, we can profile the score distribution of all experts. As depicted in Figure 6-(b), we set $T_1 = 0.6$ and $T_2 = 0.9$ for the Mixtral-8x7B model, dividing the experts into three groups: 67% in high precision, 30% in low precision, and 3% to skip. This configuration maintains model accuracy while significantly reducing expert-loading costs. Due to Mixtral-8x7B’s top-2 selection mechanism, all top-1 experts (50% of selections) receive scores of 0 and remain in the high-precision group.

With this method, MoE-APEX can dynamically load experts with the appropriate precision based on the current input when a cache miss occurs, significantly reducing expert-loading latency while maintaining both model accuracy and deployment flexibility.

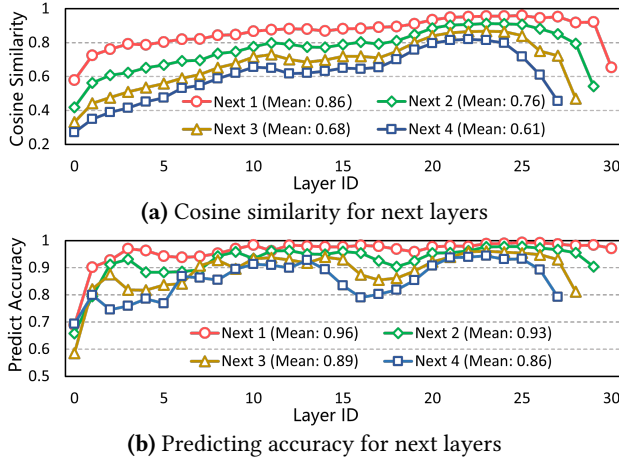


Figure 8. Cosine similarity and predicting accuracy across layers of Mixtral-8x7B, where "Next i " refers to the next i -th layer from the current layer.

3.3 Layer-level Adaptive Expert Prefetching

To fully leverage the benefits of overlapping communication with computation, we require a highly accurate method for prefetching mixed precision experts for subsequent layers. Due to the layer-by-layer structure of LLMs, we can explore the similarities between model layers to design the method.

Similarity between layers. Due to the residual structure in LLMs, hidden states across consecutive layers exhibit significant similarity [5, 29, 39]. This suggests that the inputs to the gating function in the MoE module also share high similarity across successive layers. As shown in Figure 8-(a), the cosine similarity of gating inputs between two consecutive layers (labeled as "Next 1" in the figure) is notably high in the Mixtral-8x7B model. In fact, even the inputs for the next two and three layers exhibit considerable similarity. As a result, we can leverage the gating input from the current layer to predict the required experts for subsequent layers. Figure 8-(b) demonstrates that the top-1 expert prediction accuracy for the next layer is very high, averaging 96% across layers. Even for the next two or three layers, the accuracy remains around 90% on average across all layers.

Takeaways: We can exploit the strong layer-wise similarity of gating inputs to design an accurate and efficient expert prefetching mechanism.

Expert predictor design. Based on these observations, we build the layer-level Adaptive Expert Predictor. As depicted in Figure 9, we begin by ① predicting the experts required for the next layer. If all predicted experts are present in the expert cache, we then proceed to predict for the subsequent layer. This process ② continues until either some predicted experts are missing from the cache or all predictions are completed (predicting p subsequent layers per layer).

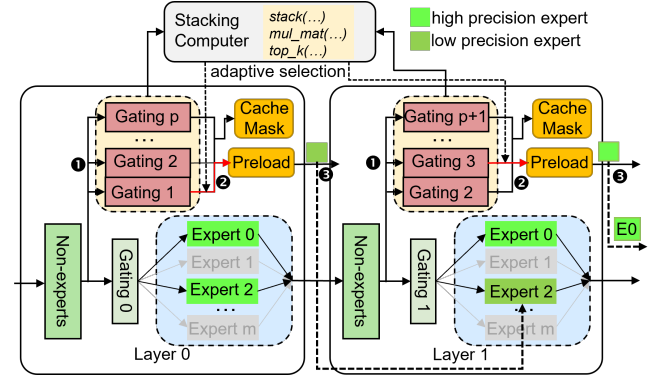


Figure 9. Layer-level Adaptive Expert Predictor.

For example, in layer 0, ③ the expert 2 for layer 1 (gating 1) need to be preloaded, while expert 0 for layer 3 (gating 3) are preloaded at layer 1 since all predicted experts for layer 2 are already in the expert cache. Furthermore, we mask all predicted experts to prevent them from being evicted from the expert cache, as they are highly likely to be used in the subsequent layers. And we preload versions of the experts with different precision levels to facilitate faster loading.

When integrating the predictor into the system, we must consider the computational overhead of the predictor. In a naive approach, the gating function would be computed sequentially until the required experts are identified, resulting in an overhead that grows linearly with the number of gating computations. Obviously, this method is inefficient. Given that one dimension of the gating module's weight corresponds to the number of experts (typically small values such as 8, 16, or 64), we can optimize the process by stacking all p gating modules together and computing them simultaneously. This approach nearly matches the computational speed of a single gating module, taking advantage of the high parallel performance offered by GPUs. Therefore, we design the Stacking Computer module to compute all p gating modules at once using several tensor operations, including *stacking*, *matrix multiplication*, and *top-k selection*, and to adaptively select the required experts for preloading. This stacking module efficiently identifies the required experts while minimizing the prediction overhead.

In addition, enables effective prefetching requires minimizing misprediction overhead. By using overlapped, block-by-block loading with immediate termination, the worst-case overhead is limited to a single weight block load (e.g., 3–4 ms for Mixtral-8x7B), a negligible penalty given the high prediction accuracy. Overall, with this predictor, MoE-APEX can fully exploit the benefits of prefetching for mixed precision expert loading.

3.4 Sequence-level Cost-aware Expert Caching

To fully leverage the potential of the mixed precision expert cache, it is crucial to design an effective cache replacement

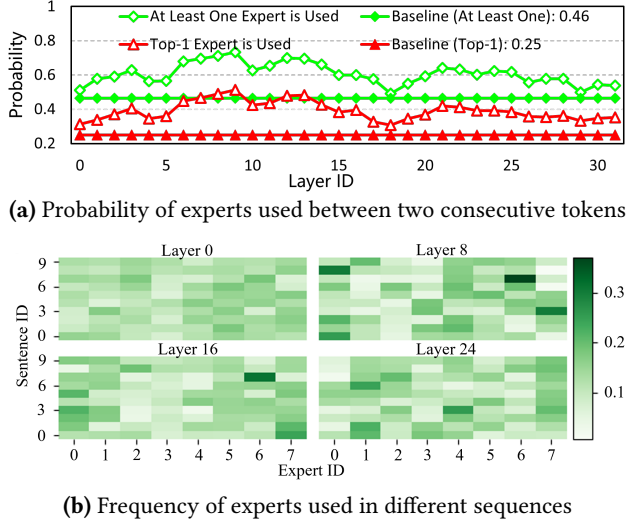


Figure 10. Statistics of experts usage for Mixtral-8x7B.

policy that accounts for the varying loading costs of low-precision and high-precision experts.

Cache replacement policies. Traditionally, Least Recently Used (LRU) and LFU methods have been employed for cache management. Previous studies [13, 27] suggest that if an expert is used in the current token’s forward pass, it has a higher probability of being utilized in the next token’s forward pass, a behavior characteristic of LRU. As illustrated in Figure 10-(a), in the Mixtral-8x7B model, the top-1 expert used in the current token process has a significantly higher likelihood of being used in the next token process than the theoretical probability of 0.25. Additionally, the probability of reusing at least one of the two experts exceeds the theoretical 0.46.

While MoE models are typically trained with an auxiliary loss to promote uniform expert selection, the frequency of expert selection varies at the sequence level. Figure 10-(b) shows that different sequences exhibit preferences for specific experts in different layers. Therefore, a sequence-level LFU can be a possible option. Furthermore, due to the layer-wise structure of these models, experts from nearer layers are more likely to be used, which we refer to as the Farthest Layer Distance (FLD) policy.

For our special mixed precision expert cache, it is necessary to define a specialized cache miss penalty rather than relying solely on the cache miss ratio to evaluate replacement policies, as experts of different precisions incur different penalties. Specifically, if an expert is missed, the cost of loading its high-precision version is C , while the cost of loading the low-precision version is only $\frac{B_l}{B_h}C$, where B_l and B_h represent the bit-widths of the low- and high-precision versions, respectively. Consequently, a new policy is needed to manage this mixed-precision scenario in order to minimize miss penalties effectively.

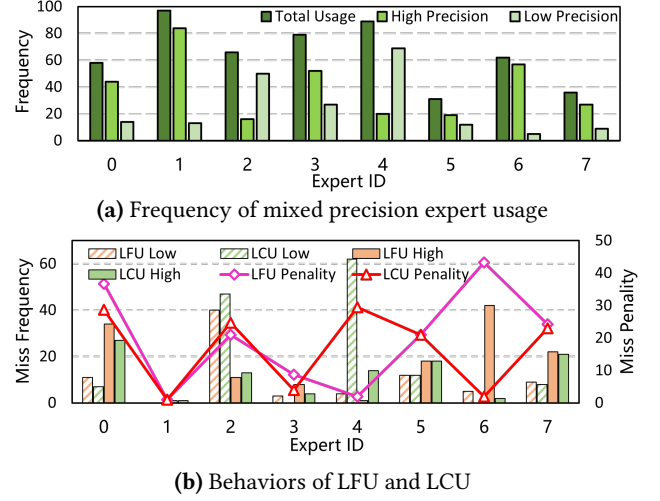


Figure 11. Mixed precision expert usage in one layer.

Takeaways: Mixed precision scenario necessitates a cost-aware caching policy that integrates the characteristics of LRU and sequence-level LFU.

Cache manager design. To minimize cache miss penalties in the mixed precision expert cache, we propose a cost-aware caching policy called Least Costly Used (LCU), which prioritizes experts that incur higher loading costs. Unlike LFU, LCU simultaneously tracks the usage frequencies of both the low- and high-precision versions of each expert. As shown in Figure 11-(a), the usage frequencies of low- and high-precision versions differ from one another and from the total usage frequency. To combine the costs of both the low- and high-precision versions, we define the cost C_t of expert t as follows:

$$C_t = H_t + \frac{B_l}{B_h}L_t \quad (3)$$

where H_t is the frequency of high-precision usage in the current sequence, and L_t is the frequency of low-precision usage. With this metric, LCU would prioritize expert 6 while LFU would prioritize expert 4 in Figure 11-(a), making LCU a distinct policy from LFU in this context.

Figure 11-(b) shows the performance comparison between LFU and LCU for these experts. The results indicate that LCU causes more cache misses for expert 4, especially for its low-precision version, while LFU keeps expert 4 in the cache with fewer misses. However, for expert 6, LCU assigns higher priority, resulting in fewer misses, especially for the high-precision version. Since expert 6 would cause higher costs due to its greater use of the high-precision version, LCU reduces cache miss penalties more effectively than LFU. Overall, for these experts, LCU reduces cache miss penalties by about 15% compared to LFU totally. Therefore, LCU is a more suitable policy in our specific scenario than LFU.

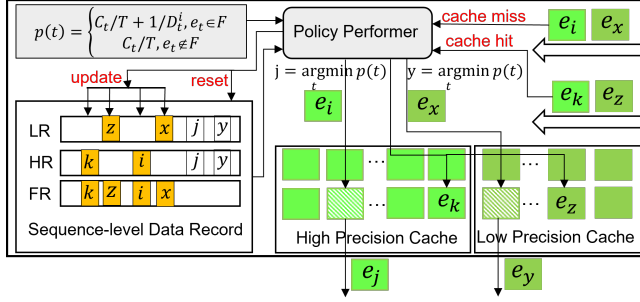


Figure 12. Sequence-level Cost-aware Cache Manager.

To leverage the observation that an expert is more likely to be used in the current forward pass if it was used in the previous one, we assign additional priority to those experts that were used in the last forward pass. Therefore, the final priority of expert t is defined as follows:

$$p_t = \begin{cases} C_t/T + \frac{1}{D_t^i}, & e_t \in F \\ C_t/T, & e_t \notin F \end{cases} \quad (4)$$

where C_t is the cost defined in Equation (3), T is the current token number used to normalize C_t , D_t^i is the layer distance of expert t to expert i , computed as $(l_t - l_i + l_n) \% l_n + \delta$. Here, l_i is the layer ID of the currently used expert i , l_t is the layer ID of expert t , l_n is the total number of layers in the model, and δ is a small constant (0.1 in our setting) to avoid division by zero when $i = t$. F is the set that contains the experts used in last forward pass.

Using this equation, we can identify the expert j with the lowest priority in the cache, relative to the current expert i , and replace j with i . As shown in Figure 12, We build the Cost-aware Cache Manager based on this equation and data records. The Cache Manager maintains separate caches for high- and low-precision experts and a sequence-level data record to store history statistics required in Equation (4). Specifically, LR records the frequency of low-precision versions, HR records the frequency of high-precision versions, and FR contains the experts used in last forward pass. Whenever a high-precision expert e_i is added to the cache (a cache miss), the Policy Performer module in Cache Manager will update HR, and FR records and determine the lowest-priority expert e_j based on Equation (4) and the data in the record. The Policy Performer then evict e_j and replace it with e_i in the high-precision cache. Similarly, for a low-precision expert e_x , the Policy Performer performs the same operation but updates the LR instead of the HR. On a cache hit, the Policy Performer only updates the relevant records (e.g., e_k and e_z in the figure). Additionally, at the start of each new sequence, the Policy Performer resets all records.

By fully leveraging the characteristics of MoE models and unique features of mixed expert cache, the Cost-aware Cache Manager can efficiently manage the cache and achieve lower

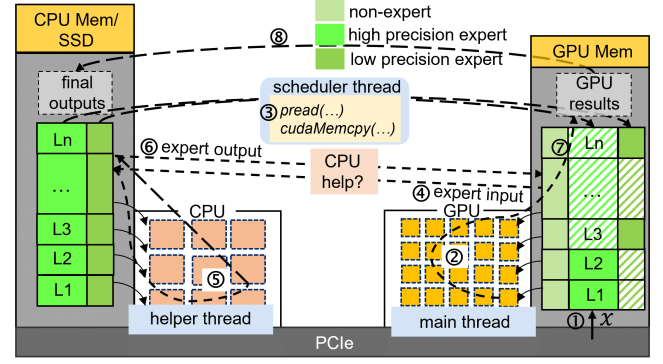


Figure 13. The implementation of MoE-APEX.

cache miss penalties than previous approaches, resulting in faster inference.

4 System Implementation

We build our system on top of Llama.cpp by modifying the distribution of model weights and computation patterns, implemented with 8,500 lines of C++/C code. The Llama.cpp system places a sufficient number of layers in GPU memory, with the remaining layers stored in CPU memory or on SSD. It processes input on the GPU using layers in GPU memory, then sends the internal activations of the last GPU-processed layer to the CPU. It continues processing with the remaining layers on the CPU, and finally gets the results. While this computation pattern works well for dense models, it is not optimal for MoE-based LLMs.

To optimize our system for MoE models, we modify the distribution of model weights. As illustrated in Figure 13, we place all non-expert weights and a portion of experts, in multiple precision versions, in GPU memory, and all expert weights reside in CPU memory or SSD. To ensure the system performs efficiently across various hardware setups, we implement two computing modes: GPU-centric computing and CPU-GPU cooperative computing.

In the GPU-centric computing mode, when ① input x is processed, the main thread ② handles it on the GPU using the corresponding model weights. If the required expert are not in GPU memory, the scheduler thread ③ loads the appropriate version of the expert from CPU memory or SSD through system interfaces. Once the required expert is loaded into GPU memory, the main thread ⑦ resumes computation and eventually ⑧ transfers the final results back to the CPU.

In the CPU-GPU cooperative computing mode, if the required expert is not in GPU memory, the main thread ④ sends the expert's input to the CPU, where a helper thread ⑤ processes it using the corresponding expert. The helper thread then ⑥ sends the expert's output back to the GPU. Once receiving the data, the main thread ⑦ continues the computation and ⑧ copies the results back to the CPU.

Table 1. Hardware setups of three tested platforms.

	Jetson Orin	RTX 4090	RTX 2080 Ti
GPU Mem.	32GB	24GB	11GB
CPU Mem./SSD	1TB	256GB	256GB
IO Speed	7GB/s	32GB/s	16GB/s
CPU Cores	12	64	40

Table 2. Configuration of evaluated MoE models.

	Mixtral-8x7B	Phi-MoE
Total Weight Size	87GB	78GB
Experts Weight Size	84GB (96%)	75GB (96%)
Layer Number	32	32
Expert Number/Layer	8	16
Top-K	2	2

	DeepSeek-MoE	DeepSeekV2-Lite
Total Weight Size	31GB	29GB
Experts Weight Size	28GB (90%)	27GB (93%)
Layer Number	28	27
Expert Number/Layer	64	64
Top-K	6	6

While both computing modes work well for MoE models, we primarily focus on the GPU-centric computing mode, as CPU resources are typically insufficient on edge devices and are usually required by other processes.

5 Experimental Evaluation

5.1 Experimental Methodology

Hardware. To evaluate MoE-APEX in different environments, we use three common edge devices: the NVIDIA Jetson AGX Orin [42], the NVIDIA GeForce RTX 4090 [41], and the NVIDIA GeForce RTX 2080 Ti [40]. As shown in Table 1, the Jetson Orin has 32GB of unified memory shared with its 12 CPU cores. For model weight storage, we use a Samsung NVMe SSD 980 PRO, which provides 1TB of storage with a theoretical read speed of 7,000 MB/s (approximately 3,000 MB/s in practice). The RTX 4090 has 24GB of GPU memory, 256GB of CPU memory, and 64 CPU cores. The connection between the CPU and GPU uses PCIe 4.0, offering a theoretical bandwidth of 32GB/s. The RTX 2080 Ti is equipped with 11GB of GPU memory, 256GB of CPU memory, and 40 CPU cores. Its CPU-GPU connection uses PCIe 3.0, with a theoretical bandwidth of 16GB/s.

Models. We evaluate our system using four popular MoE-based LLMs from Huggingface Hub [14]: Mixtral-8x7B [27], Phi-MoE [1], DeepSeek-MoE [9], and DeepSeekV2-Lite [10]. As summarized in Table 2, Mixtral-8x7B employs 8 experts per layer with 2 experts activated per token, while Phi-MoE uses 16 experts per layer, also activating 2 experts per token. In contrast, the two DeepSeek models follow a more recent

Table 3. Speed (tokens/s) under different sample number.

	10	100	1,000	10,000	50,000
Mixtral-8x7B	2.23	2.26	2.26	2.25	2.25
Phi-MoE	6.09	6.15	6.08	6.14	6.10
DeepSeek-MoE	9.07	9.24	9.28	9.25	9.27
DeepSeekV2-Lite	9.40	9.85	9.98	9.94	9.94

trend in MoE design, employing a larger number of experts per layer and a higher number of activated experts per token. **Datasets.** To efficiently evaluate inference speed, we select 60 high-quality samples from the 52k-sample Alpaca dataset [49] as our speed test set. As reported in Table 3, varying the number of test samples has a negligible effect on measured decoding speed, validating the use of this small subset for performance comparisons. To evaluate the impact of MoE-APEX on model accuracy, we use GSM8K [8], TruthfulQA [36] and ARC [7] as performance evaluation datasets. GSM8K is designed to evaluate a model’s mathematical reasoning capabilities. TruthfulQA assesses whether a language model can generate factually accurate responses. And ARC measures a model’s common sense reasoning abilities.

Baselines. We compare MoE-APEX (MA) with seven SOTA inference systems to evaluate its efficiency. (1) Transformers [55] (TF), a general LLM library developed by Huggingface, offering thousands of pretrained models. (2) DeepSpeed-Inference [3] (DS), a comprehensive inference system for LLMs, providing multi-GPU and heterogeneous inference solutions. (3) Llama.cpp [18] (LL), an efficient LLM inference system written in pure C/C++, supporting simultaneous computation on both CPU and GPU. (4) MoE-Offloading [13] (MO), a MoE-centric system that incorporates expert prediction and caching. (5) MoE-Infinity [58] (MI), a system that tracks request-level processes to prefetch required experts into GPU memory. (6) AdapMoE [66] (AM), an adaptive system that skips some unimportant experts to accelerate inference. (7) Fiddler [28] (FD), a system that leverages CPU to process experts existed in CPU memory for minimizing data movement between the CPU and GPU.

Configurations. Due to platform differences, we use different configurations to evaluate baselines. On the RTX 4090, we employ Mixtral-8x7B and Phi-MoE with float16 precision. Since Llama.cpp and Fiddler utilize CPU computation, which follows a different computational pattern from other methods, we compare them separately for fairness. On the Jetson Orin, we use the int8 precision versions of Mixtral-8x7B and Phi-MoE, as the float16 versions are too large and slow to run due to the SSD’s slow read speed. And we only evaluate Llama.cpp and MoE-Infinity on the Jetson Orin, as the other baselines don’t support this device well. On the RTX 2080 Ti, we evaluate DeepSeek-MoE and DeepSeekV2-Lite using

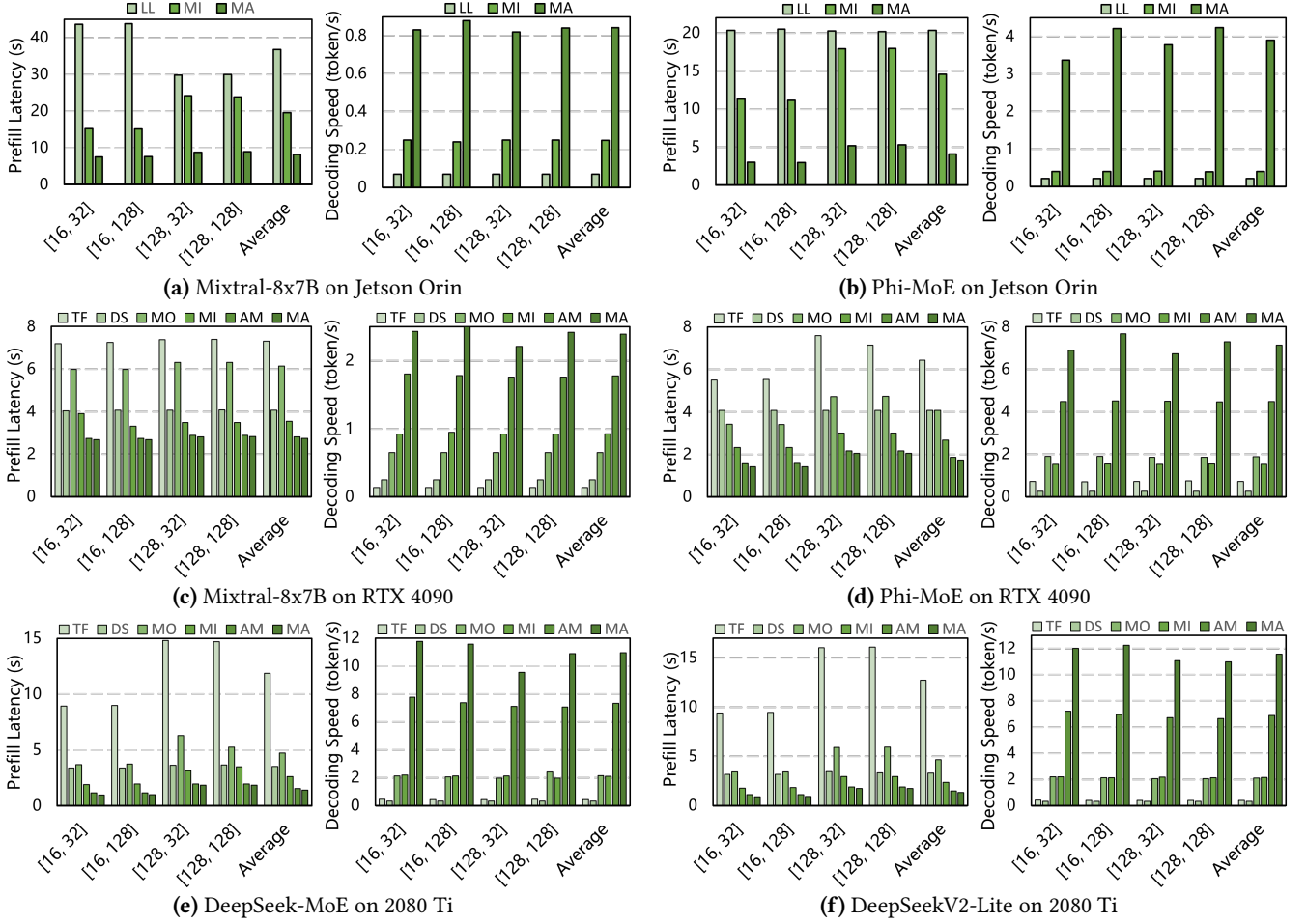


Figure 14. Comparison of inference speed for MoE-APEX and the SOTA approaches.

float16 precision to assess system performance under configurations with a large number of experts. Furthermore, for MoE-APEX, we use int2 precision versions as replacements for both the float16 precision models and the int8 precision models to support dynamic precision expert loading.

Metrics. Since the generation process of LLMs consists of two phases (the prefill stage and the decoding stage), we use prefill latency (in seconds) and decoding speed (in tokens per second) as our performance metrics. To strengthen and diversify the results, we set four testing groups with different input and output lengths, including [16, 32], [16, 128], [128, 32], [128, 128]. And we set the batch size to 1 in all cases, following prior works [13, 25, 30, 60], as edge-side continuous serving scenarios often focus on single-batch inference.

5.2 End-to-End Performance

To evaluate the inference speed of MoE-APEX, we conduct the experiments described before and obtain the results shown in Figure 14.

From Figure 14-(a) and (b), we can observe that MoE-APEX delivers the best performance in terms of both decoding speed and prefill latency for the evaluated models on the Jetson AGX Orin, outperforming both Llama.cpp and MoE-Infinity. Llama.cpp utilizes the *mmap()* interface for fast model loading, but this can result in severe page faults when there is insufficient CPU memory to store the model weights. Since Jetson AGX Orin shares memory between the CPU and GPU, allocating memory for GPU computation leaves limited memory available for the CPU, leading to performance degradation due to frequent page faults. MoE-Infinity, primarily designed for GPU servers, also faces challenges on Jetson AGX Orin due to the slower SSD read speeds. Compared to Llama.cpp, MoE-APEX achieves an average speedup of 12.0× for Mixtral-8x7B and 18.57× for Phi-MoE in decoding speed, along with a 78% and 80% reduction in prefill latency for these two models, respectively. Although MoE-APEX is built on Llama.cpp, it shows significantly greater advantages when deploying MoE-based LLMs on edge-embedded devices. In comparison to MoE-Infinity, MoE-APEX delivers

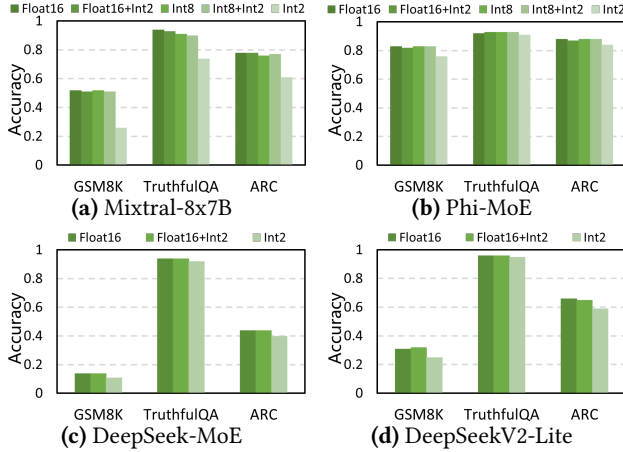


Figure 15. Model accuracy with mixed precision experts.

an average speedup of $3.36\times$ and $9.75\times$ in decoding speed, along with a 58% and 72% reduction in prefill latency for these two evaluated models, respectively. These improvements are largely attributed to its dynamic precision expert loading mechanism, which reduces the volume of data to be read, thereby accelerating inference speed.

On the GeForce RTX 4090, the results shown in Figure 14-(c) and (d) demonstrate that MoE-APEX again outperforms the other baselines. Transformers and DeepSpeed-Inference show poor performance compared to MoE-based systems, as they do not leverage the sparse activation feature of MoE and load model parameters layer by layer on demand. MoE-Offloading and MoE-Infinity exhibit varying performance in decoding speed depending on the model, with MoE-Infinity performing better on Mixtral-8x7B, while MoE-Offloading excels with Phi-MoE. However, MoE-Infinity consistently achieves lower prefill latency than MoE-Offloading for its good prefetching technique. While both methods surpass Transformers and DeepSpeed-Inference, they are hindered by high expert-loading latency. For example, loading two experts can take 20 ms, dwarfing the 3.4 ms required for computation (Figure 4-(a)). In contrast, AdapMoE improves performance over MoE-Offloading and MoE-Infinity by skipping unimportant experts, thereby reducing the number of experts that need to be loaded from CPU memory. However, compared to AdapMoE, MoE-APEX achieves higher decoding performance by not only skipping experts but also replacing them with lower-precision versions. This combination of skipping and replacement provides MoE-APEX with greater flexibility in reducing loading latency, whereas AdapMoE’s skip-only strategy can lead to significant accuracy degradation when too many experts are skipped. Specifically, compared with AdapMoE, MoE-APEX delivers an average decoding speedup of $1.34\times$ for Mixtral-8x7B and $1.59\times$ for Phi-MoE, along with prefill latency reductions of 2% and 7%.

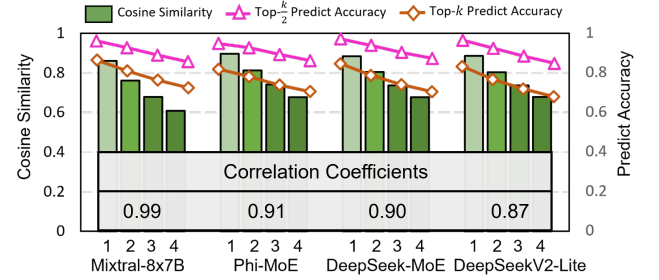


Figure 16. Correlation coefficients, cosine similarity and predict accuracy for all models, where 1, 2, 3, and 4 indicate the next one, two, three, and four layers, respectively.

To verify that MoE-APEX remains effective with large MoE experts, we benchmark it on the RTX 2080 Ti with two DeepSeek models, which have 64 experts per layer. As shown in Figures 14-(e) and (f), MoE-APEX consistently outperforms all baselines in both prefill latency and decoding speed. Compared to the best baseline, AdapMoE, MoE-APEX achieves a 9% reduction in prefill latency and a $1.49\times$ speedup in decoding speed for DeepSeek-MoE, and an 11% reduction and a $1.68\times$ speedup for DeepSeekV2-Lite. These results demonstrate that MoE-APEX still works well with modern MoE architectures featuring a large number of experts.

Overall, our system achieves significant speedup compared to baselines, demonstrating its superior efficiency.

5.3 Model Accuracy

Since MoE-APEX replaces high-precision experts with low-precision versions to accelerate expert loading, it is essential to verify that accuracy is not compromised. As shown in Figure 15, we report the average accuracy on GSM8K and ARC, along with the informative score for TruthfulQA. After applying the mixed-precision expert policy (the Float16+Int2 and Int8+Int2), accuracy drops by no more than 1% across all evaluated models and datasets in the three test groups of Figure 14. These results demonstrate that MoE-APEX achieves high inference speed while preserving model accuracy.

5.4 Model Characteristic Analysis

To verify that all evaluated models share similar characteristics with Mixtral-8x7B, as previously described in Section 3, we analyze their key properties. First, as shown in Figure 16, the correlation coefficients follow the same trend observed in Figure 6-(a), confirming that all MoE models maintain high correlation levels similar to Mixtral-8x7B. This validates the reliability of our metric for assessing expert importance. Second, all models achieve high prediction accuracy and strong cosine similarity between layers, again aligning with the performance of Mixtral-8x7B. These results collectively demonstrate that our design generalizes well across different MoE models, confirming its robustness and applicability.

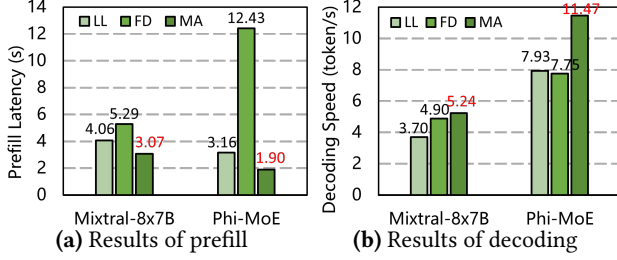


Figure 17. Inference speed for RTX 4090 + CPU setup.

5.5 CPU Computation Assistant

Although we primarily focus on common GPU-centric scenarios, sometimes there are enough CPU resources to help speed up the computation. To accommodate such case, we implement a CPU-GPU cooperative computing mode to enhance system flexibility and maximize hardware resource utilization. We evaluate this computing mode using the RTX 4090 + CPU setup on Mixtral-8x7B and Phi-MoE.

From the results in Figure 17, we can observe that MoE-APEX consistently outperforms Llama.cpp in both decoding speed and prefill latency. Specifically, compared to Llama.cpp, MoE-APEX delivers a 1.42× and 1.45× speedup in decoding speed, along with a 24% and 40% reduction in prefill latency for the two evaluated models, respectively.

Fiddler demonstrates some advantages over Llama.cpp when running on Mixtral-8x7B, but exhibits similar decoding performance on Phi-MoE. However, Fiddler’s prefill latency increases exponentially with the number of experts, as it explores all possible combinations (2^n , where n is the number of experts per layer) to identify the optimal configuration for partitioning expert processing between the CPU and GPU. This becomes a performance bottleneck on Phi-MoE, where the number of experts is double that of Mixtral-8x7B, resulting in significantly higher prefill latency. Consequently, Fiddler is better suited for specific use cases and struggles to scale across different models and environments. In contrast, compared to Fiddler, MoE-APEX achieves a 1.07× and 1.48× speedup in decoding speed, and a 42% and 85% reduction in prefill latency for the two evaluated models, respectively.

Therefore, although CPU-GPU cooperation is not the primary use case for MoE-APEX, it still provides better performance than current systems.

5.6 Ablation Study

5.6.1 Dynamic Expert Loading Mechanism Analysis.

To evaluate the benefits of the dynamic expert loading mechanism, we test models across all setups and averaged the results for different input and output lengths. As shown in Figure 18, using the dynamic loading mechanism provides a speedup ranging from 1.22× to 1.53× under different configurations. The highest speedup is observed when running

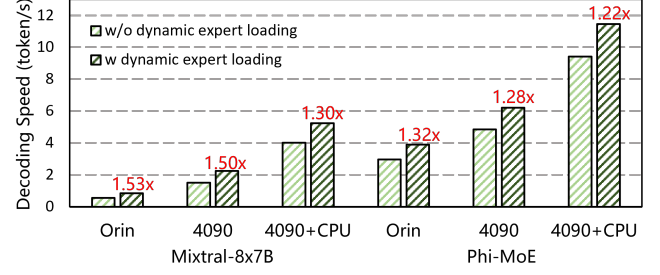


Figure 18. Inference speedup of dynamic expert loading.

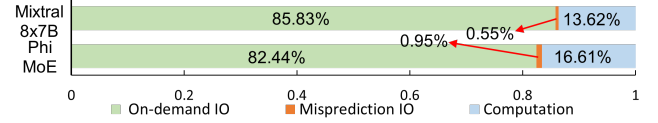


Figure 19. Breakdown of inference latency (normalized).

models on the Jetson Orin device, which is due to its relatively slower data transfer speeds compared to the RTX 4090. Conversely, the lowest speedup is seen on the RTX 4090 assisted by the CPU, as in this setup, the performance gains primarily stem from CPU computing improvements with low-precision experts, rather than faster expert loading. Additionally, we observe that the Mixtral-8x7B model achieves a greater speedup than the Phi-MoE model, likely due to the larger expert sizes in the Mixtral-8x7B model. Overall, these findings suggest that the dynamic expert loading mechanism is especially advantageous in environments with slower data transfer speeds and for models with larger experts.

5.6.2 Adaptive Expert Prefetching Technique Analysis.

We begin by evaluating the stacking operation in our design. As shown in Figure 20-(a), the sequential operation time increases linearly as the number of predicting layers grows, whereas our stacking operation remains stable, validating our previous analysis. Next, we assess the benefits of the adaptive expert prefetching technique. Experiments were conducted on the RTX 4090 setup, comparing performance with and without dynamic expert loading. As illustrated in Figure 20-(b), the prefetching technique significantly improves performance during the prefill stage, reducing prefill latency by approximately 10% in all cases. During the decoding stage, the prefetching technique performs better when combined with dynamic expert loading (Float16+Int2). Specifically, without dynamic expert loading (Float16), the prefetching technique provides modest performance improvements, achieving a 1.05× speedup for Mixtral-8x7B and a 1.02× speedup for Phi-MoE. In contrast, with dynamic expert loading, the prefetching technique achieves a 1.10× speedup for Mixtral-8x7B and a 1.15× speedup for Phi-MoE. This demonstrates that the prefetching technique

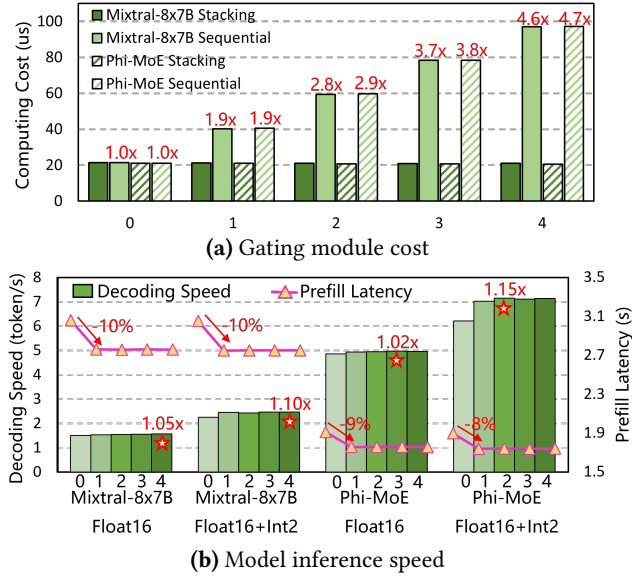


Figure 20. Gating module cost and model inference speed with different configurations of the adaptive predictor, where 0 means no prediction, and 1, 2, 3, and 4 represent predictions for 1, 2, 3, and 4 subsequent layers, respectively.

provides greater benefits when combined with dynamic expert loading. We also find that the parameter p influences performance and our experiments suggest an optimal range of 2–4. Furthermore, we breakdown the inference latency for $p = 2$ on the speed test set with input and output lengths of [16, 32]. As shown in Figure 19, expert loading (IO) dominates latency, accounting for over 80% in both models, whereas computation contributes less than 20%, consistent with prior observations. Moreover, the overhead (Misprediction IO) of misprediction is very small (less than 1%), far smaller than the performance benefits gained from prefetching. Overall, these results confirm the efficiency of the proposed adaptive prefetching technique.

5.6.3 Cost-aware Expert Caching Policy Analysis. To verify the robustness of our proposed cache replacement policy, we compared it with other commonly used policies across different settings. As shown in Figure 21-(a), LCU consistently achieves the lowest cache miss penalty (normalized against the random policy baseline, with the reduction compared to LFU highlighted, as LFU performs best among the baselines). Traditionally, LRU performs best under most cache conditions, but it performs poorly in this case due to the sequential execution mode of the layer-by-layer structure in LLMs. LFU outperforms LRU in this context, showing a reduction of 2.49% ~ 4.80% compared to LRU in cache miss penalties. Overall, LCU results in a 4.85% ~ 7.88% reduction in cache miss penalties compared to LRU and a 2.36% ~ 3.10% reduction compared to LFU. Additionally, we compared the model-level and sequence-level performance

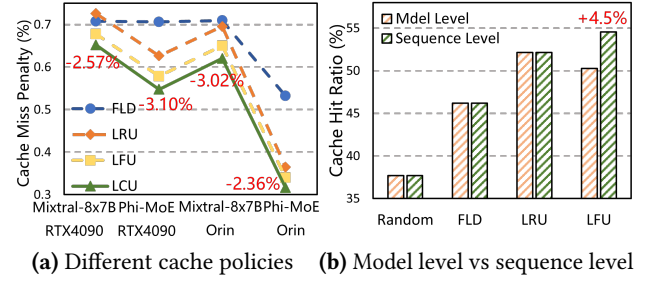


Figure 21. Cache performance with different strategies.

Table 4. Memory overhead for evaluated models.

Model	Total Size	Expert Size	Overhead
Mixtral-8x7B	87 GB	84 GB	10.5 GB (12%)
Phi-MoE	78 GB	75 GB	9.4 GB (12%)
DeepSeek-MoE	31 GB	28 GB	4.7 GB (15%)
DeepSeekV2-Lite	29 GB	27 GB	4.5 GB (16%)

of different policies, as shown in Figure 21-(b). The sequence-level performance particularly affects LFU policies, where sequence-level LFU achieves a 4.5% increase in hit ratio. This behavior is expected, as the balanced loss function in the training process of MoE-based LLMs encourages an equal frequency of expert selection in a big dataset, as discussed earlier. In contrast, other policies perform similarly across both model-level and sequence-level conditions. Therefore, a sequence-level policy is particularly suitable for MoE when considering the LFU-based strategies.

5.7 Overhead Analysis

MoE-APEX dynamically replaces high-precision experts with low-precision experts, which require additional CPU memory for storage. Since Int2 precision is employed, this memory overhead amounts to only 1/8 of the corresponding Float16 expert size. As shown in Table 4, the maximum memory overhead among the evaluated models does not exceed 11 GB, which is considered acceptable given that available CPU memory often extends to hundreds of gigabytes. The two DeepSeek models exhibit a slightly higher overhead ratio because their *down_proj* weights are quantized using Int4 precision. This was necessary because the small dimensions of this particular matrix are not supported by the Int2 quantization implementation in Llama.cpp. Therefore, this overhead is not a fundamental limitation and is expected to be eliminated as quantization techniques advance.

6 Related Work

Expert-offloading systems for MoE-based LLMs. Given the significant GPU memory requirements of MoE-based

LLMs, many systems have been developed to optimize inference on memory-constrained devices by offloading expert parameters to CPU memory or SSD. Some systems, such as fMoE [61], ProMoE [46], MoE-Offloading [13], MoE-Infinity [58], Pre-gated MoE [25], and SwapMoE [30], focus on optimizing prefetching techniques and cache replacement policies. Others, like EdgeMoE [60] and AdapMoE [66], aim to reduce expert-loading costs. Additionally, approaches such as Fiddler [28] and DAOP [64] leverage CPU computational power to assist in the inference process. However, these systems do not fully exploit the key characteristics of MoE-based LLMs, resulting in large expert-loading costs or big drops in model accuracy on edge devices.

Optimized inference systems for LLMs. The large size and complexity of LLMs have driven the development of various inference systems aimed at optimizing the efficiency and performance of the inference process [54, 56]. Some of these works focus on adaptive offloading, such as Vllm [31], DeepSpeed [3], Accelerate [20] and Llama.cpp [18], which speed up inference by offloading model weights or internal activations to alternative hardware, such as CPU memory or SSDs. Other works focus on exploiting the sparsity in LLMs to reduce computational cost and memory consumption by skipping the computation of inactive neurons. Examples include Deja Vu [37], PowerInfer [47, 59], and CATS [32]. However, these systems have primarily been designed to target dense LLMs, where all model parameters are utilized uniformly across different inputs. As a result, they often fail to exploit the unique characteristics of MoE-based LLMs [22].

Model compression techniques for LLMs. Model compression is a promising area of research that offers various techniques to effectively deploy LLMs in resource-constrained environments. The core idea of model compression is to reduce the size of a large model while preserving its performance. Popular compression techniques, such as model quantization [11, 16, 52, 57], network pruning [35, 38, 39, 48], knowledge distillation [2, 19, 23], and low-rank factorization [34, 53, 62], enable the creation of more resource-efficient LLMs. However, these methods only focus on algorithmic optimization, so they often overlook the practical challenges of deploying LLMs on resource-constrained devices, leading to inefficient deployments.

7 Conclusion

In this work, we introduce MoE-APEX, a flexible and efficient inference system for deploying MoE LLM models on memory-constrained edge devices. By addressing the high cost associated with expert loading in existing MoE inference systems, MoE-APEX enables significant speedups in inference performance. Additionally, we have integrated MoE-APEX into the Llama.cpp inference framework. This integration allows for flexible scalability across a wide range of platforms. Overall, MoE-APEX represents a significant step

forward in making complex MoE models more accessible and practical for edge computing environments.

Acknowledgments

We sincerely thank all the anonymous reviewers for their valuable comments and feedback. This work was supported by the National Natural Science Foundation of China (No. U24A20234), the Young Elite Scientists Sponsorship Program by CAST (No. YESS20240529) and the Research Grants Council of the Hong Kong SAR, China, under Project T45-401/22-N. Xiaofeng Hou and Chao Li are the corresponding authors.

References

- [1] Marah Abdin, Sam Ade Jacobs, Ammar Ahmad Awan, Jyoti Aneja, Ahmed Awadallah, Hany Awadalla, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Harkirat Behl, et al. 2024. Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219* (2024).
- [2] Rishabh Agarwal, Nino Vieillard, Piotr Stanczyk, Sabela Ramos, Matthieu Geist, and Olivier Bachem. 2023. Gkd: Generalized knowledge distillation for auto-regressive sequence models. *arXiv preprint arXiv:2306.13649* (2023).
- [3] Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, and Yuxiong He. 2022. DeepSpeed-Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–15.
- [4] Apple. 2024. Introducing Apple's On-Device and Server Foundation Models. <https://machinelearning.apple.com/research/introducing-apple-foundation-models>.
- [5] Xiaodong Chen, Yuxuan Hu, and Jing Zhang. 2024. Compressing large language models by streamlining the unimportant layer. *arXiv preprint arXiv:2403.19135* (2024).
- [6] Xiangxiang Chu, Limeng Qiao, Xinyu Zhang, Shuang Xu, Fei Wei, Yang Yang, Xiaofei Sun, Yiming Hu, Xinyang Lin, Bo Zhang, et al. 2024. Mobilevlm v2: Faster and stronger baseline for vision language model. *arXiv preprint arXiv:2402.03766* (2024).
- [7] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge. *arXiv:1803.05457v1* (2018).
- [8] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168* (2021).
- [9] Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, Zhenda Xie, Y. K. Li, Panpan Huang, Fuli Luo, Chong Ruan, Zhifang Sui, and Wenfeng Liang. 2024. DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models. *CoRR abs/2401.06066* (2024).
- [10] DeepSeek-AI. 2024. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. *arXiv:2405.04434* [cs.CL].
- [11] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. Gpt3. int8 (): 8-bit matrix multiplication for transformers at scale. *Advances in Neural Information Processing Systems* 35 (2022), 30318–30332.
- [12] David Eigen, Marc'Aurelio Ranzato, and Ilya Sutskever. 2013. Learning Factored Representations in a Deep Mixture of Experts. *CoRR*

- abs/1312.4314 (2013).
- [13] Artyom Eliseev and Denis Mazur. 2023. Fast inference of mixture-of-experts language models with offloading. *arXiv preprint arXiv:2312.17238* (2023).
 - [14] Hugging Face. 2024. Hugging Face Hub. <https://huggingface.co/docs/hub/index>.
 - [15] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch transformers: scaling to trillion parameter models with simple and efficient sparsity. *J. Mach. Learn. Res.* 23, 1, Article 120 (jan 2022), 39 pages.
 - [16] Elias Frantar, Saleh Ashkboos, Torsten Hoeftler, and Dan Alistarh. 2022. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323* (2022).
 - [17] Othmane Friha, Mohamed Amine Ferrag, Burak Kantarci, Burak Cakmak, Arda Ozgun, and Nassira Ghoulami-Zine. 2024. LLM-Based Edge Intelligence: A Comprehensive Survey on Architectures, Applications, Security and Trustworthiness. *IEEE Open Journal of the Communications Society* (2024).
 - [18] Georgi Gerganov. 2023. ggerganov/llama.cpp: Port of facebook's llama model in c/c++. <https://github.com/ggerganov/llama.cpp>.
 - [19] Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. 2023. Knowledge distillation of large language models. *arXiv preprint arXiv:2306.08543* (2023).
 - [20] Sylvain Gugger, Lysandre Debut, Thomas Wolf, Philipp Schmid, Zachary Mueller, Sourab Mangrulkar, Marc Sun, and Benjamin Bossan. 2022. Accelerate: Training and inference at scale made simple, efficient and adaptable. <https://github.com/huggingface/accelerate>.
 - [21] J.B. Hampshire and A. Waibel. 1992. The Meta-Pi network: building distributed knowledge representations for robust multisource pattern recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 14, 7 (1992), 751–769. doi:10.1109/34.142911
 - [22] Wei Huang, Yue Liao, Jianhui Liu, Ruifei He, Haoru Tan, Shiming Zhang, Hongsheng Li, Si Liu, and Xiaojuan Qi. 2024. Mixture Compressor for Mixture-of-Experts LLMs Gains More. *arXiv preprint arXiv:2410.06270* (2024).
 - [23] Yukun Huang, Yanda Chen, Zhou Yu, and Kathleen McKeown. 2022. In-context learning distillation: Transferring few-shot learning ability of pre-trained language models. *arXiv preprint arXiv:2212.10670* (2022).
 - [24] Huawei. 2023. Beating Google and Apple, Huawei brings large AI model to mobile voice assistant. <https://www.huaweicentral.com/beating-google-and-apple-huawei-brings-large-ai-model-to-mobile-voice-assistant/>.
 - [25] Ranggi Hwang, Jianyu Wei, Shijie Cao, Changho Hwang, Xiaohu Tang, Ting Cao, and Mao Yang. 2024. Pre-gated moe: An algorithm-system co-design for fast and scalable mixture-of-expert inference. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 1018–1031.
 - [26] Robert Jacobs, Michael Jordan, Steven Nowlan, and Geoffrey Hinton. 1991. Adaptive Mixture of Local Expert. *Neural Computation* 3 (02 1991), 78–88. doi:10.1162/neco.1991.3.1.79
 - [27] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. 2024. Mixtral of experts. *arXiv preprint arXiv:2401.04088* (2024).
 - [28] Keisuke Kamahori, Yile Gu, Kan Zhu, and Baris Kasikci. 2024. Fidler: CPU-GPU Orchestration for Fast Inference of Mixture-of-Experts Models. *arXiv preprint arXiv:2402.07033* (2024).
 - [29] Bo-Kyeong Kim, Geonmin Kim, Tae-Ho Kim, Thibault Castells, Shinkook Choi, Junho Shin, and Hyoung-Kyu Song. 2024. Shortened llama: A simple depth pruning for large language models. *arXiv preprint arXiv:2402.02834* (2024).
 - [30] Rui Kong, Yuanchun Li, Qingtian Feng, Weijun Wang, Xiaozhou Ye, Ye Ouyang, Linghe Kong, and Yunxin Liu. 2024. SwapMoE: Serving Off-the-shelf MoE-based Large Language Models with Tunable Memory Budget. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 6710–6720.
 - [31] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626.
 - [32] Je-Yong Lee, Donghyun Lee, Genghan Zhang, Mo Tiwari, and Azalia Mirhoseini. 2024. CATS: Contextually-Aware Thresholding for Sparsity in Large Language Models. *arXiv preprint arXiv:2404.08763* (2024).
 - [33] Jiamin Li, Qiang Su, Yitao Yang, Yimin Jiang, Cong Wang, and Hong Xu. 2023. Adaptive gating in mixture-of-experts based language models. *arXiv preprint arXiv:2310.07188* (2023).
 - [34] Pingzhi Li, Zhenyu Zhang, Prateek Yadav, Yi-Lin Sung, Yu Cheng, Mohit Bansal, and Tianlong Chen. 2023. Merge, then compress: Demystify efficient SMoe with hints from its routing policy. *arXiv preprint arXiv:2310.01334* (2023).
 - [35] Yixiao Li, Yifan Yu, Qingru Zhang, Chen Liang, Pengcheng He, Weizhu Chen, and Tuo Zhao. 2023. Lospars: Structured compression of large language models based on low-rank and sparse approximation. In *International Conference on Machine Learning*. PMLR, 20336–20350.
 - [36] Stephanie Lin, Jacob Hilton, and Owain Evans. 2021. Truthfulqa: Measuring how models mimic human falsehoods. *arXiv preprint arXiv:2109.07958* (2021).
 - [37] Zichang Liu, Jue Wang, Tri Dao, Tianyi Zhou, Binhang Yuan, Zhao Song, Anshumali Shrivastava, Ce Zhang, Yuandong Tian, Christopher Re, et al. 2023. Deja vu: Contextual sparsity for efficient llms at inference time. In *International Conference on Machine Learning*. PMLR, 22137–22176.
 - [38] Xinyin Ma, Gongfan Fang, and Xinchao Wang. 2023. Llm-pruner: On the structural pruning of large language models. *Advances in neural information processing systems* 36 (2023), 21702–21720.
 - [39] Xin Men, Mingyu Xu, Qingyu Zhang, Bingning Wang, Hongyu Lin, Yaojie Lu, Xianpei Han, and Weipeng Chen. 2024. Shortgpt: Layers in large language models are more redundant than you expect. *arXiv preprint arXiv:2403.03853* (2024).
 - [40] NVIDIA. 2019. GeForce RTX 2080 Ti. https://www.nvidia.com/content/force-gtx/GEFORCE RTX_2080Ti_User_Guide.pdf.
 - [41] NVIDIA. 2024. GeForce RTX 4090. <https://www.nvidia.com/en-us/force/graphics-cards/40-series/rtx-4090/>.
 - [42] NVIDIA. 2024. Jetson AGX Orin Developer Kit. <https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-orin/>.
 - [43] Samyam Rajbhandari, Olatunji Ruwase, Jeff Rasley, Shaden Smith, and Yuxiong He. 2021. Zero-infinity: Breaking the gpu memory wall for extreme scale deep learning. In *Proceedings of the international conference for high performance computing, networking, storage and analysis*. 1–14.
 - [44] Qualcomm AI Research. 2023. World's first on-device demonstration of Stable Diffusion on an Android phone. <https://www.qualcomm.com/news/onq/2023/02/worlds-first-on-device-demonstration-of-stable-diffusion-on-android>.
 - [45] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538* (2017).
 - [46] Xiaoni Song, Zihang Zhong, Rong Chen, and Haibo Chen. 2024. Promoe: Fast moe-based llm serving using proactive caching. *arXiv preprint arXiv:2410.22134* (2024).
 - [47] Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. 2023. Powerinfer: Fast large language model serving with a consumer-grade gpu. *arXiv preprint arXiv:2312.12456* (2023).
 - [48] Mingjie Sun, Zhuang Liu, Anna Bair, and J Zico Kolter. 2023. A simple and effective pruning approach for large language models. *arXiv*

- preprint arXiv:2306.11695* (2023).
- [49] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. Stanford Alpaca: An Instruction-following LLaMA model. https://github.com/tatsu-lab/stanford_alpaca.
 - [50] Qwen Team. 2024. Qwen1.5-MoE: Matching 7B Model Performance with 1/3 Activated Parameters". <https://qwenlm.github.io/blog/qwen-moe/>
 - [51] Volker Tresp. 2000. Mixtures of Gaussian Processes. In *Advances in Neural Information Processing Systems*, T. Leen, T. Dietterich, and V. Tresp (Eds.), Vol. 13. MIT Press.
 - [52] Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Huaijie Wang, Lingxiao Ma, Fan Yang, Ruiping Wang, Yi Wu, and Furu Wei. 2023. Bitnet: Scaling 1-bit transformers for large language models. *arXiv preprint arXiv:2310.11453* (2023).
 - [53] Xin Wang, Yu Zheng, Zhongwei Wan, and Mi Zhang. 2024. Svd-llm: Truncation-aware singular value decomposition for large language model compression. *arXiv preprint arXiv:2403.07378* (2024).
 - [54] Xinkai Wang, Yiming Zhuansun, Chao Li, Jing Wang, Xiaofeng Hou, Lingyu Sun, Luping Wang, and Minyi Guo. 2025. Asymserve: Demystifying and optimizing llm serving efficiency on cpu acceleration units. In *International Symposium on Advanced Parallel Processing Technologies*. Springer, 231–245.
 - [55] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. 2020. Transformers: State-of-the-Art Natural Language Processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, Online, 38–45.
 - [56] Feiyang Wu, Zhuohang Bian, Guoyang Duan, Tianle Xu, Junchi Wu, Teng Ma, Yongqiang Yao, Ruihao Gong, and Youwei Zhuo. 2025. TokenSim: Enabling hardware and software exploration for large language model inference systems. In *International Symposium on Advanced Parallel Processing Technologies*. Springer, 257–266.
 - [57] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning*. PMLR, 38087–38099.
 - [58] Leyang Xue, Yao Fu, Zhan Lu, Luo Mai, and Mahesh Marina. 2024. Moe-infinity: Activation-aware expert offloading for efficient moe serving. *arXiv preprint arXiv:2401.14361* (2024).
 - [59] Zhenliang Xue, Yixin Song, Zeyu Mi, Le Chen, Yubin Xia, and Haibo Chen. 2024. PowerInfer-2: Fast Large Language Model Inference on a Smartphone. *arXiv preprint arXiv:2406.06282* (2024).
 - [60] Rongjie Yi, Liwei Guo, Shiyun Wei, Ao Zhou, Shangguang Wang, and Mengwei Xu. 2023. Edgemoe: Fast on-device inference of moe-based large language models. *arXiv preprint arXiv:2308.14352* (2023).
 - [61] Hanfei Yu, Xingqi Cui, Hong Zhang, and Hao Wang. 2025. fMoE: Fine-Grained Expert Offloading for Large Mixture-of-Experts Serving. *arXiv preprint arXiv:2502.05370* (2025).
 - [62] Zhihang Yuan, Yuzhang Shang, Yue Song, Qiang Wu, Yan Yan, and Guangyu Sun. 2023. Asvd: Activation-aware singular value decomposition for compressing large language models. *arXiv preprint arXiv:2312.05821* (2023).
 - [63] Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. 2024. TinyLlama: An Open-Source Small Language Model. [arXiv:2401.02385](https://arxiv.org/abs/2401.02385) [cs.CL]
 - [64] Yujie Zhang, Shivam Aggarwal, and Tulika Mitra. 2025. DAOP: Data-Aware Offloading and Predictive Pre-Calculation for Efficient MoE Inference. In *2025 Design, Automation & Test in Europe Conference (DATE)*. IEEE, 1–7.
 - [65] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2023. A survey of large language models. *arXiv preprint arXiv:2303.18223* (2023).
 - [66] Shuzhang Zhong, Ling Liang, Yuan Wang, Runsheng Wang, Ru Huang, and Meng Li. 2024. AdapMoE: Adaptive sensitivity-based expert gating and management for efficient moe inference. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*. 1–9.